

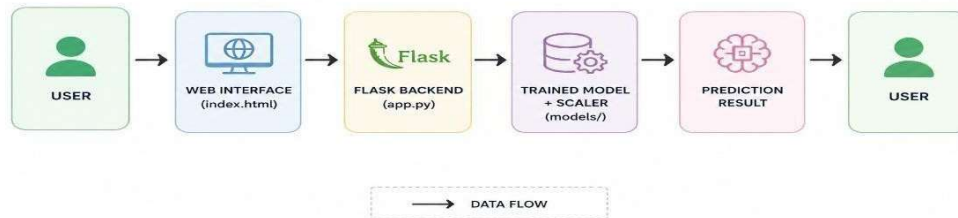
Project Title	Rising Waters: Machine Learning Based Flood Prediction System		
Developer	M. Thrishna Tanmayee	Team ID	[Team ID]
Date	26 June	Phase / Document	Requirement Analysis

5. Data Flow Diagram

The data flow for Rising Waters is intentionally simple and linear: there is a single ingestion point (the prediction form), a single processing pipeline (scale → predict → classify → explain), and a single output surface (the same page, re-rendered with results). This section documents that flow at two levels of detail — a context-level view of the major actors, and a step-by-step process view that mirrors the actual code path inside app.py.

5.1 Level 0 — Context Diagram

DATA FLOW DIAGRAM



User → Web Interface (index.html) → Flask Backend (app.py) → Trained Model + Scaler (models/) → Prediction Result → User.

At this level, the system has exactly one external actor (the user) and one internal boundary worth noting: the trained model and scaler are produced entirely offline by train.py and are only ever read, never modified, by the live Flask process. This keeps the request-serving path stateless and fast, since no training or fitting happens while a request is being handled.

5.2 Level 1 — Process Flow

- User submits weather parameters via the form on index.html, or triggers the Auto-Fill generator for demo values.
- Flask route /predict receives form values via request.form and casts each field to float into an input_dict.
- Values are assembled into a single-row pandas DataFrame (features_df) with column names matching the training schema.
- The pre-fitted StandardScaler (scaler.joblib) transforms the DataFrame into scaled features.
- The trained classifier (flood_model.joblib) runs predict() and predict_proba() on the scaled features.
- The probability of the flood class is converted to a percentage and mapped to a severity tier: ≥ 75% Severe Flood Warning, ≥ 40% Elevated Flood Watch, else Normal Conditions.
- Rule-based logic generates human-readable insights (e.g. rainfall, humidity, cloud cover thresholds) and recommended protocols matching the severity tier.
- Results are passed back into index.html via render_template and displayed to the user.

Because every step above runs synchronously within a single Flask request, the entire round trip — from form submission to rendered result — completes in one request/response cycle with no background jobs, queues, or polling required. This keeps the diagram, and the underlying implementation, deliberately shallow.